

Lab 1

Performance Evaluation of Computer Systems and Networks

MSc in Computer Engineering

MSc in Artificial Intelligence and Data Engineering

Academic Year 2026/2027

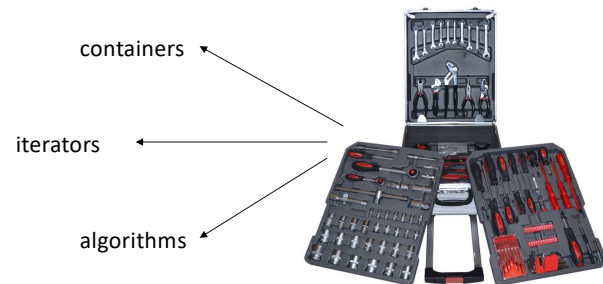
Dr. Ing. Giovanni Nardini



UNIVERSITÀ DI PISA

The Standard Template Library (STL)

- Do not reinvent the wheel
- Develop faster
- Using standard components
 - Safer (less error prone)
 - Possibly higher efficiency



https://www.cppreference.com/cpp/standard_library

Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved

STL provides a **library** for you to build programs involving elements such as **containers** (data structures), **iterators** (pointers) and **algorithms**. It is called standard “template” library because such tools can be used with any type of data (integers, real numbers, strings, complex structures...)

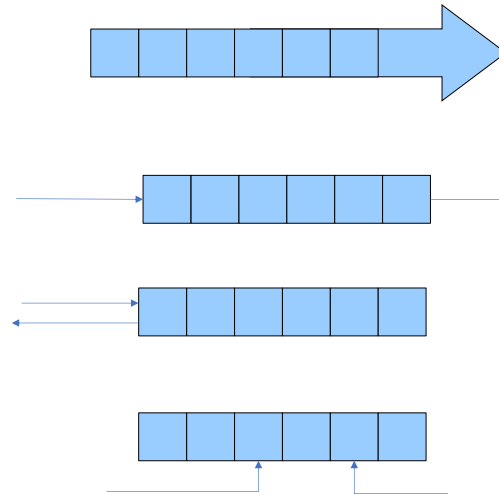
STL is very well documented: [see cppreference.com](https://www.cppreference.com)

STL can be used as a tool for speeding up code writing. Also, it is “safer”, as it has been widely tested and verified. This means that there is a smaller risk that they cause errors.

Moreover, they generally come with complexity bounds, that you can translate into performance bounds.

Example: STL containers

**Multiple
data
structures**



Extendable

FIFO

LIFO

Direct access

Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved

STL provides you with multiple **data structures**.

Data structures (and the ways you access them) are a crucial building block in any piece of software. Different data structures behave differently with different operations.

It is not important to know exactly the behavior of each STL container, but rather understanding the general principles (C++ reference manual is extremely helpful).

std::vector

```
#include <vector>
using namespace std;
```

```
vector<int> stlArray;
```

Container type

Variable name

Element type

amortized constant complexity

`stlArray.push_back(val)`



`stlArray[i]`

constant complexity

Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved

The complexity/cost of the `push_back` operation is:

“Constant (amortized time, reallocation may happen). If a reallocation happens, the reallocation is itself up to linear in the entire size.”

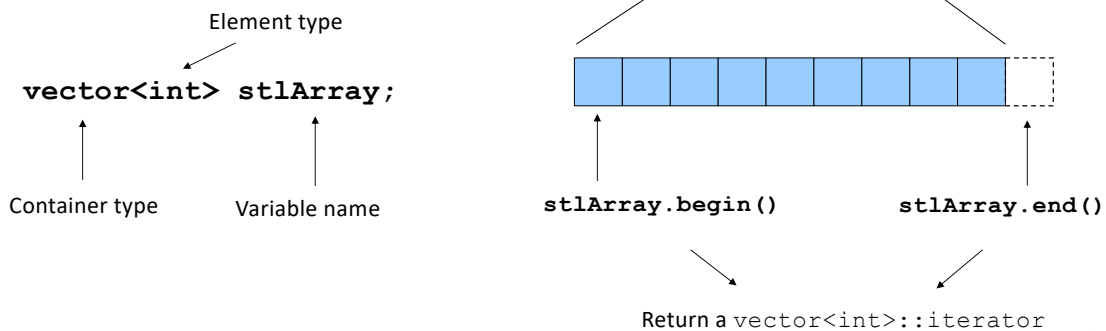
(https://www.cplusplus.com/reference/vector/vector/push_back/)

Different implementations may take different approaches to reach this goal. Example: creating another vector with double the size, and copying the elements.

The overall idea is that, in the typical case (in a statistical sense) the operation has a **constant cost**. In rare cases it might cost more (at worst linear cost in the entire size).

std::vector

```
#include <vector>
using namespace std;
```



Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved

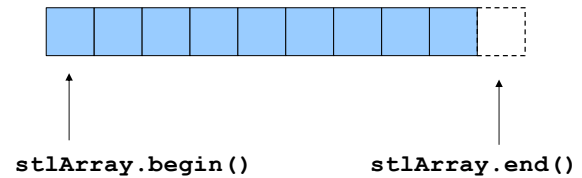
The `end()` function returns an iterator to the past-the-end element in the vector container.

An **iterator** is an object (like a pointer) that points to an element inside the container, and can be used to move through the contents of the container. They can be visualized as something similar to *pointers* pointing to some location, and we can access the content at that particular location through them.

Why not use pointers? For a vector you could, since its elements are contiguous. But in other containers (lists, maps, sets) elements are not stored contiguously, so pointer arithmetic does not work. Iterators give the same interface for every container, which lets STL algorithms like `sort` work on any of them and lets you change container with minimal code changes.

Iterators

```
#include <vector>
using namespace std;
```



```
vector<int> stlArray;
...
vector<int>::iterator it;
for(it = stlArray.begin(); it != stlArray.end(); ++it) {

    int n = (*it);
    ...
}
```

Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved

Use the **++ operator** on the iterator to move through the container.
Use the *** operator** to access the elements «pointed» by the iterator.

Sorting a container

```
#include <vector>
#include <algorithm>
using namespace std;

sort( stlArray.begin(), stlArray.end() );

sort( stlArray.begin()+2, stlArray.end() );
```

$$\Theta(n \log n)$$

Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved

`sort` is an example of **algorithm** provided by the STL, which sorts the elements in the range `[first,last)` into ascending order.

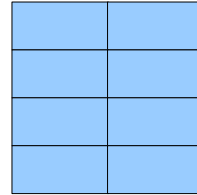
std::map

```
#include <map>
using namespace std;
```

```
map < key_type, T > table;
```

Type of the key

Data type




```
map<string, int> table;
table["one"] = 1;
map<string, int>::iterator it = table.find("one");
int val = table["one"];    // same as int val = *it
```

Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved

Sorted associative container that contains **key-value pairs with unique keys**. While vectors allow you to access an element using the element index (as in a traditional array), with maps you can access it using its key.

Search, removal, and insertion operations have logarithmic complexity.

std::priority_queue

```
#include <queue>
using namespace std;
```

- Provides **constant time lookup of the largest element...**
- ...at the expense of **logarithmic insertion and extraction**

```
priority_queue<int> prioQ;
int val = 5;
prioQ.push(val);           // insert an element (and sort the underlying container)
val = prioQ.top();         // returns the largest element
prioQ.pop();               // extracts (remove) the largest element
```

This container implements a **max heap** data structure.

In case you need a different type of «priority», you need to implement your own comparator.

std::priority_queue

- You can change the policy, by defining a **comparator** function

```
#include <queue>

struct Comp {
    bool operator()(int a, int b) {
        if (a > b) return true;
        else      return false;
    }
};

priority_queue<int, vector<int>, Comp> prioQ;
...
int val = prioQ.top();    // returns the smallest element
```

Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved

What if we want to implement a min heap?

When you insert an element in the container, the **comparator** function is invoked to do pairwise comparison between the new element and the elements in the container, in order to find the correct position for the new element.

The correct position is found when the comparator function returns false.

The Comp object is called a functor, that is an **object that behaves like a function**, i.e. can be invoked like you do with a function, and it executes a custom code defined by the programmer by overloading the operator().

Choose the proper data structure

- Functional point-of-view
- Performance point-of-view
 - General case
 - Depending on your data
- Evaluate alternatives (trade offs)
 - Performance analysis
 - Search for critical cases
- Always keep in mind the developing time

Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved

All the considerations on complexity analysis are still valid, but...

Remember you should take into account worst, best and average cases, also depending on your data. Example: insertion sort: good with nearly-sorted numbers, but it is bad in the worst-case scenario.

Take development time into account. If a new solution that barely improves performance requires a huge amount of time for development, it might not be a good overall solution.

Extra: measuring the execution time

- Insert «probes» in your code

```
#include <sys/time.h>
...
struct timeval tStart, tStop;
gettimeofday(&tStart, NULL);
...
gettimeofday(&tStop, NULL);
double time = (tStop.tv_sec * 1000 + tStop.tv_usec / 1000)
              - (tStart.tv_sec * 1000 + tStart.tv_usec / 1000);
```

Unix

```
#include <windows.h>
...
unsigned int tStart, tStop;
tStart = GetTickCount();
...
tStop = GetTickCount();
double time = tStop - tStart;
```

Windows

- In Unix systems you can use the `time` command in the shell (duration of the whole program)

```
$ time ./my_prog
```

```
real    0m0,804s
user    0m0,804s
sys     0m0,000s
```

Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved

GetTickCount returns a value in milliseconds

Alternative: use a code profiler, such as Valgrind.

Hands on session

Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved

Exercise 1

Create a C++ program that defines and uses a new class named `EventQueue`.

`EventQueue` includes an **unordered array** of real numbers and supports the following operations:

- `insert(n)` inserts the number given as argument at the end of the array
- `next()` returns the number with lowest value and removes it from the array. If the array is empty, the function returns `-1`

Hint: use `std::vector` to implement the array within the `EventQueue` class

Hint2: `std::vector` provides the `erase()` function to remove an element from the container

Write a `main` function that:

- Instantiates one object of type `EventQueue`, call it `q`
- Reads real numbers from the keyboard, until the user types a negative number. All the numbers read must be inserted into `q` by calling the `insert()` function
- By using the `next()` function, extracts and prints out in ascending order all the numbers in `q`

Exercise 2

Modify `EventQueue` so that its elements are stored in a **min-heap** data structure. Modify the `insert()` and `next()` functions accordingly

Hint: use `std::priority_queue` allocator with a custom comparator

Then, do the following:

1. Run the program by using the file *“random.txt”* as input (find the file on the Teams channel)
 - **Hint:** launch the program from the command line using input redirection via the operator `'<'`
 - `./ex2 < random.txt`
2. Measure the execution time of the program by using the command **time**
 - `time ./ex2 < random.txt`
3. Measure the execution time of the program of exercise 1 by using the command **time**
 - `time ./ex1 < random.txt`

Do you notice any difference?

Exercise 3

Modify `EventQueue` so that its elements are stored in a **calendar queue** with $M = 1000$ buckets of width $\delta = 10$. Modify the `insert()` and `next()` functions accordingly

Hint: use a `std::vector` of M buckets, and implement each bucket using either `std::vector` or `std::list` keeping its elements sorted by firing time

Then:

1. Run the program with “*random.txt*” as input and measure its execution time
 - `time ./ex3 < random.txt`
2. Compare it with the execution times of exercises 1 and 2
3. Repeat step 1 with $M = 10$ and $M = 10000$. ***Do you notice any difference? Why?***

Extension: replace the `std::list` of each bucket with a min-heap and re-evaluate the execution times

Exercise 1 – hints

```
#include <iostream>
#include <vector>
using namespace std;

class EventQueue {
    vector<float> queue_;

public:
    EventQueue() {}
    ~EventQueue() {}

    void insert(const float& f);
    float next();
};

void EventQueue::insert(const float& f)
{
}

float EventQueue::next()
{
}

int main() {

    return 0;
}
```

Exercise 2 – hints

```
#include <iostream>
#include <vector>
#include <queue>
...
class EventQueue {
    struct Comparator {
        bool operator()(const float& f1, const float& f2) {

        }
    };
    priority_queue<float, vector<float>, Comparator> queue_;
public:
    ...
};
```

Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved

Exercise 3 – hints

```
#include <iostream>
#include <vector>
#include <list>

...
class EventQueue {
    const unsigned int M = 1000;
    const float D = 10;

    vector< list<float> > queue_; // M sorted lists

    unsigned int currentBucket_; // index of the current bucket
    unsigned int currentYear_;   // index of the current calendar year
    unsigned int size_;          // number of stored events
public:
    ...
};
```

Lab 1

Ing. Giovanni Nardini - University of Pisa - All rights reserved